金融平台接口文档 V1.0

金融平台接口文档

1.1 使用账号密码登录，获取token

接口地址: `/app-api/system/third/login`

请求方式: `POST`

请求数据类型: `application/json`

响应数据类型: `*/*`

接口描述:

请求示例:

```
{
  "username": "test2025",
  "password": 123456,
  "sign": "",
  "timestamp": 0,
  "appId": ""
}
```

请求参数:

参数名称	参数说明	请求类型	是否必须	数据类型
username	账号		true	string
password	密码		true	string
sign	签名		true	string
timestamp	时间戳		true	integer(int64)
appId	应用id		true	string

响应状态:

状态码	说明
200	OK
非200	登录失败，查看msg获取原因

响应参数:

参数名称	参数说明	类型
code		integer(int32)
msg		string
data		
userId	用户编号	integer(int64)
accessToken	访问令牌	string

响应示例:

```
{
    "code": 0,
    "msg": "",
    "data": {
        "userId": 1024,
        "accessToken": "happy"
    }
}
```

签名计算规则:

新增用户以后，会生成一对公钥私钥，同时存在 appId。

将 username, password, timestamp(当前时间戳), appId 按照字典序排序，使用 & 进行拼接，然后通过私钥进行加密获取到sign。

注意：时间戳有效期5分钟。

示例：

```
appId=ronsafe&password=ronsafe123&timestamp=1766643951234&username=rongx
```

上述字符串通过 私钥 进行加密，得到 sign。
python加密函数参考

```
import hashlib
import base64
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256

def sign(data: str, base64_private_key: str) -> str:
"""
基于Crypto包（pycryptodome）实现
```

```python
    :param data: 待签名的明文字符串
    :param base64_private_key: Base64编码的PKCS8格式私钥
    :return: Base64编码的签名字符串
    """
        try:

            # 1. 解码Base64私钥
            key_bytes = base64.b64decode(base64_private_key.strip())

            # 2. 导入PKCS8格式的RSA私钥
            rsa_private_key = RSA.import_key(key_bytes)

            # 3. 处理待签名文本：转为UTF-8字节流
            text_bytes = data.encode('utf-8')

            # 4. 计算待签名文本的SHA256哈希（SHA256withRSA的哈希环节）
            sha256_hash = SHA256.new(text_bytes)

            # 5. 执行RSA签名：使用PKCS#1 v1.5填充
            signed_bytes = pkcs1_15.new(rsa_private_key).sign(sha256_hash)

            # 6. 编码输出：Base64编码签名字节流
            base64_sign = base64.b64encode(signed_bytes).decode('utf-8')

            return base64_sign

        except Exception as e:
            raise Exception(f"签名失败: {str(e)}")
```

1.2 登出系统

接口地址: `/app-api/system/third/logout`

请求方式: `POST`

接口描述: 登出并注销登录状态

请求Header:

名称	内容
Authorization	Bearer {上面获取到的token值，去掉此处的大括号}

请求参数:

暂无

响应状态:

状态码	说明
200	OK

响应参数:

参数名称	参数说明	类型
code		integer(int32)
msg		string
data		boolean

响应示例:

```
{
    "code": 0,
    "msg": "",
    "data": true
}
```

1.3 批量下APP指标任务

接口地址: `/app-api/system/third/checkAppTaskBatch`

请求方式: `POST`

请求数据类型: `application/json`

响应数据类型: `*/*`

接口描述: 用户端批量下任务

请求Header:

名称	内容
Authorization	Bearer {上面获取到的token值，去掉此处的大括号}

请求示例:

```
[
  {
    "idType": "010105",
    "exid": ""
  }
```

```
]
```

请求参数:

参数名称	参数说明	请求类型	是否必须	数据类型
thirdTaskParams	三方账号下任务参数	body	true	array
idType	参数类型		true	string
exid	查询内容		true	string

参数说明:

idType	对应的类型
010105	PHONE
010207	IMEI
010208	IMSI
010209	MAC
010213	IDFA
010214	OAID

响应状态:

状态码	说明
200	OK

响应参数:

参数名称	参数说明	类型
code		integer(int32)
msg		string
data	生成的任务id	string

响应示例:

```
{
    "code": 0,
    "msg": "",
    "data": 1234556678
```

```
}
```

1.4 查询APP指标任务结果

接口地址: `/app-api/system/third/queryAppTaskResult`

请求方式: `GET`

请求数据类型: `application/x-www-form-urlencoded`

响应数据类型: `*/*`

接口描述: 根据1.3 生成的任务 ID 查询任务详情

请求Header:

名称	内容
Authorization	Bearer {上面获取到的token值，去掉此处的大括号}

请求参数:

参数名称	参数说明	请求类型	是否必须	数据类型
taskId		query	true	string

响应状态:

状态码	说明
200	请求成功
204	任务在进行中

响应参数:

参数名称	参数说明	类型
code		integer(int32)
msg		string
data	响应结果，结果为加密文本，解密的key为login接口返回的userid。解密后为json格式内容	string

响应示例:

```
{
    "code": 0,
    "msg": "",
    "data": ""
}
```

python解密函数参考代码

```python
import hashlib
import base64
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad

def decrypt(cipher_text: str, user_id: str) -> str:
    try:
        # 生成 AES key，取 SHA-256 的前 16 字节
        sha256 = hashlib.sha256(user_id.encode('utf-8')).hexdigest()
        key = sha256[:16].encode('utf-8')
        # AES 默认使用 ECB 模式，Java Hutool 的 AES 默认是 ECB + PKCS5Padding
        cipher = AES.new(key, AES.MODE_ECB)
        # 解密前需要 base64 解码
        encrypted_bytes = base64.b64decode(cipher_text)
        # 解密并去掉 padding
        decrypted_bytes = unpad(cipher.decrypt(encrypted_bytes), AES.block_size)
        return decrypted_bytes.decode('utf-8')

    except Exception as e:
        print(f"decrypt failed: {e}")
        raise ServiceException(f"数据解密失败: {e}")
```

data结构说明：

```
{
  "13016850000": [ // 手机号（key）
    {
      "byOdsDeviceInfo": [ // 该手机号绑定的设备信息
        {
          "id": "13016850000", // 手机号
          "idType": "010105", // ID类型
          "deviceId": "2e30e********", // 设备号
          "deviceTime": "2019-06-09",// 设备首次关联时间
          "deviceLastTime": "2021-12-17",// 设备最后一次关联时间
          "cellFactory": "HUAWEI",// 设备品牌
          "model": "P10",// 设备型号
          "languageName": "",// 设备语言
          "sysver": "9.0",// 系统版本号
```

```
15            "oaid": "53ECEB14",
16            "idfa": "",
17            "imeiList": "[\"86292*********\"]",// 绑定的imei列表
18            "macList": "[]",
19            "phoneList": "[{\"carrier\":\"中国移动\",\"code\":\"18702170000\",\"firstTime\":\"2023-08-03 00:00:00\",\"lastTime\":\"2023-08-03 00:00:00\",\"location\":\"上海-上海\",\"status\":\"实号\"},{\"carrier\":\"中国联通\",\"code\":\"13016850000\",\"firstTime\":\"2019-06-09 21:19:42\",\"lastTime\":\"2021-03-18 13:01:00\",\"location\":\"江苏-常州\",\"status\":\"实号\"}]", // 绑定的手机号列表，包含首次末次绑定时间，归属地
20            "imsiList": "[{\"code\":\"460011*******\",\"firstTime\":\"2021-12-17 11:29:47\",\"lastTime\":\"2021-12-17 11:29:47\"}]" //imsi列表
21          }
22        ],
23        "debtorJson": { //计算后的二级指标结果
24          "fga": 0,
25          "gea": 5.046,
26          "mr": 64.098,
27          "rsh": 44.583,
28          "afp": 24.303,
29          "lef": 36.368,
30          "deviceId": "2956226*********",
31          "swp": "{\"health_focus_score\":0.0,\"work_scene_tag\":\"unknown\"}",
32          "prp": 32.392,
33          "br": 44.536,
34          "dnti": 74.098,
35          "labr": 83.333,
36          "originalExid": "13016850000",
37          "geaRaw": 65.036,
38          "id": "135425********2",
39          "ws": 57.274,
40          "cwp": "day_noon",
41          "ec": 35.607,
42          "rp": 65.036
43        }
44      }
45    ]
46  }
```